

IT 3

COUR DE BASES DE DONNEES (vol3)

Par : Ahmed SEREME

Diplômé d'Etudes Supérieures Spécialisé en Conception de systèmes d'information

Email seremeah@yahoo.fr

Tel 60539362

2025

DEVELOPPEMENT DE BD

Oracle

Introduction

- **Plan**
 - **Concurrences d'accès et Schémas**
 - **Le Langage PL/SQL**
 - Outils de développement de bases de données

CHAPITRE N° 1

CONCURRENCES D'ACCÈS ET SCHÉMAS

TR : I. GESTION DE LA CONCURRENCE D'ACCES

• I.1. Transaction

Définition

Une transaction (unité logique de traitement) est une séquence d'instructions SQL qui doivent s'exécuter comme un tout.

Début et fin d'une transaction Oracle

Une transaction *débute* :

- à la connexion à un outil
- à la fin de la transaction précédente.

Une transaction SQL *se termine* :

- par un ordre COMMIT ou ROLLBACK
- par un ordre du LDD valide (CREATE, DROP, RENAME, ALTER, ...)

Les clauses de contrôle du déroulement des transactions sont :

COMMIT [WORK]

SAVEPOINT savepoint_id

ROLLBACK [WORK] [TO savepoint_id]

TR : I. GESTION DE LA CONCURRENCE D'ACCES

• I.1. Transaction

Les clauses de contrôle du déroulement des transactions sont :

COMMIT [WORK]

SAVEPOINT savepoint_id

ROLLBACK [WORK] [TO savepoint_id]

COMMIT :

- valide l'ensemble des modifications depuis le début de la transaction
- libère les verrous

ROLLBACK :

- restitue les données à leur valeur de début de transaction
- libère les verrous

ROLLBACK TO SAVEPOINT :

1. Pose d'étiquette : SAVEPOINT nom
- 2 . Annulation partielle :
ROLLBACK TO [SAVEPOINT] nom

TR : I. GESTION DE LA CONCURRENCE D'ACCES

- **I.1. Transaction**

Exemple

```
create table Stock(CODE_STOCK varchar2(10),  
STOCK_QUANT number,  
DESIGNATION_STOCK VARCHAR2(30), constraint  
Pk1_Stock Primary key(CODE_STOCK));
```

```
insert into stock values('TelPort', 4, 'Téléphonie Portable');  
insert into stock values('SmartPh',10, 'Téléphonie nouvelle generation' );  
insert into stock values('laptop', 10, 'Ordinateur portables');  
insert into stock values('desktop', 5, 'Ordinateur de bureau');  
INSERT INTO stock values('Café', 10, 'Nescafé');  
INSERT INTO stock values('Cons_Sucre', 10, 'Confiture');
```

```
SQL> INSERT INTO stock values('Lait', 19, 'Laitier');
```

```
SQL> SAVEPOINT updt_conf1;
```

```
SQL> UPDATE stock SET DESIGNATION_STOCK='lait_poudre ' WHERE  
DESIGNATION_STOCK='Laitier';
```

```
SQL> ROLLBACK TO updt_conf1;
```

TR : I. GESTION DE LA CONCURRENCE D'ACCES

- **I.1. Transaction**

Exemple

```
INSERT INTO stock values('Pot_sucre', 'Confiture', 19);  
COMMIT ;  
SAVEPOINT updt_conf1;  
UPDATE stock SET DESIGNATION_STOCK='Huiles' WHERE  
DESIGNATION_STOCK = 'Confiture';  
ROLLBACK ;  
SAVEPOINT updt_conf2  
ROLLBACK TO updt_conf1 ;
```


TR : I. GESTION DE LA CONCURRENCE D'ACCES

• I.2. Les Verrous

Pour assurer l'accès aux données sans risque d'anomalie de lecture ou de mise à jour, Oracle utilise la technique du verrouillage.

Les verrous permettent d'éviter des interactions entre des utilisateurs qui accèderaient à la même ressource.

Les granules de verrouillage sont : la **table** ou la **ligne**. La pose des verrous s'effectuent de deux façons :

- **implicitement** : c'est le moteur Oracle qui décide de poser un verrou ;
- **explicitement** : c'est le programmeur qui pose explicitement les verrous.

implicitement, un verrou exclusif est posé lors d'un ordre :

- INSERT ou
- UPDATE ou
- DELETE ou
- SELECT ... FOR UPDATE

TR : I. GESTION DE LA CONCURRENCE D'ACCES

- **I.2. Les Verrous**

Verrous par défaut

X : Verrou EXCLUSIF

S: Verrou SHARE

RS: Verrou ROW SHARE
(SHARE UPDATE)

RX: Verrou ROW EXCLUSIVE

SRX: Verrou SHARED ROW
EXCLUSIVE.

Commande SQL	Verrous ligne	Mode de verrouillage de la table
SELECT	--	--
INSERT	oui	RX
UPDATE	oui	RX
DELETE	oui	RX
SELECT ... FOR UPDATE	oui	RS
LOCK table IN ROW SHARE MODE	--	RS
LOCK table IN ROW EXCLUSIVE MODE	--	RX
LOCK table IN SHARE EXCLUSIF MODE	--	SRX
LOCK table IN SHARE MODE	--	S
LOCK table IN EXCLUSIVE MODE	--	X
DDL / DCL	--	X

TR : I. GESTION DE LA CONCURRENCE D'ACCES

• I.2. Les Verrous

Verrous table (TM) :

- Si une transaction pose un verrou ligne exclusif, il est
- également posé automatiquement un verrou table sur la
- table concernée. (SELECT ... FOR UPDATE ou, - LOCK TABLE)

CLAUSE

LOCK TABLE <liste_de_table>

IN <type_de_verrou>

MODE [NOWAIT]

Exemple :

T1 (B) : LOCK TABLE CLIENT IN EXCLUSIVE MODE ;

T2 (A) : LOCK TABLE CLIENT IN SHARED UPDATE NOWAIT ;

TR : I. GESTION DE LA CONCURRENCE D'ACCES

- **I.2. Les Verrous**

Verrous au niveau ligne

- Les commandes SELECT FOR UPDATE, INSERT, DELETE, UPDATE placent un verrou exclusif sur une ou plusieurs lignes d'une table.
- Au niveau table, un verrou RS (row share) est posé pour la commande SELECT FOR UPDATE et un verrou RX (row exclusif). Un verrou ligne est toujours posé implicitement.

Exemple : SELECT * FROM STOCK
WHERE QUANTITE_STOCK=10
FOR UPDATE OF DESIGNATION_STOCK ;

TR : I. GESTION DE LA CONCURRENCE D'ACCES

- **I.2. Les Verrous**

Choix du type de verrou

- Le moteur Oracle pose des verrous implicites. Ceux-ci peuvent être remis en cause par l'utilisateur grâce aux commandes SELECT FOR UPDATE ou LOCK TABLE.
- ne poser de verrous exclusifs sur une table que si le traitement l'exige ;
- poser les verrous de préférence au niveau ligne pour favoriser la concurrence ;
- poser un verrou share (S) pour favoriser les applications en lecture.

TR : I. GESTION DE LA CONCURRENCE D'ACCES

- **I.3. Notion de séquences et d'index (dans le schémas)**

Indexes

CREATE INDEX | UNIQUE INDEX <Nom_index> **ON** <Nom table> (<Liste des champs>)

Séquences

CREATE SEQUENCE <Nom sequence>
INCREMENT BY <Increment>
START WITH <Valeur de début>
MAXVALUE <Valeur maximal>
MINVALUE <Valeur minimal>
NOCYCLE / CYCLE
CACHE <partie réservée>;

SE : II. Le Schéma externe (les vues)

• I.3. Le Schéma externe (les vues)

- Une vue est une table logique qui permet l'accès aux données de une ou plusieurs tables de bases de façon transparente.
- Une vue ne contient aucune ligne. Les données sont stockées dans les tables

Rôle:

- assurer l'indépendance logique/externe ;
- fournir un niveau supplémentaire de sécurité sur les tables de base.
- masquer la complexité : une vue peut être la jointure de N-tables ;
- fournir une nouvelle vision de la base. Au lieu de changer le nom des tables de base, on changera seulement au niveau d'une vue si le changement ne concerne pas toute les applications ;
- masquer les bases de données distantes.

SE : II. Le Schéma externe (les vues)

- **I.3. Le Schéma externe (les vues)**
- **Les vues système**

Les tables dynamiques

- **seg\$** : segments définis dans la base de données
- **obj\$** : objets définis sur la base de données
- **tab\$** : tables définies dans la base de données y compris les clusters
- **ind\$** : index définis dans la base de données
- **col\$** : colonnes définies dans la base de données
- **ts\$** : tablespaces définis dans la base de données

Les vues

- **accessible_tables** : contient les tables et vues accessibles par l'utilisateur
- **all_catalog** : tables, vues, synonym, ... accessibles par le user
- **all_tab_columns** : synonyme de la table accessible_table
- **all_tables** : description des tables accessibles par un user
- **all_users** : informations sur l'ensemble des users d'une base
- **all_views** : textes des vues accessibles par un utilisateur
- **dba_catalog** : toutes les tables, les vues, synonymes et séquences de la base

SE : II. Le Schéma externe (les vues)

- **I.3. Le Schéma externe (les vues)**
- **Les vues système**

Les vues

- **dba_data_files**: informations sur les fichiers de la base de données
 - **dba_free_space** : espace restant libre dans les fichiers de la base
 - **dba_users** : informations sur l'ensemble des users de la base
 - **dba_tables** : informations sur l'ensembles des tables de la base
 - **dba_views** : texte de toutes les vues de la base
 - **user_catalog** : tables, vues, ... dont l'utilisateur est propriétaire
 - **user_free_space** : Extent libre dans un tablespace accessible par le user
 - **user_indexes** : Descriptions des index de l'utilisateur
 - **user_tables** : tables dont l'utilisateur est propriétaire
 - **user_views** : textes des vues de l'utilisateur
 - **user_users** : info sur le user courant
 - **ALL_*** sont des vues donnant des informations sur les objets auxquels l'utilisateur a accès
 - **DBA_*** sont des vues sur tous les objets de la base
- les vues commençant par dba_ sont accessibles par le DBA

SE : II. Le Schéma externe (les vues)

- **I.3. Le Schéma externe (les vues)**

- **Les vues personnelles**

- **Syntaxe :**

CREATE [OR REPLACE]

[FORCE | NOFORCE] **VIEW** nom_de_vue

[(alias_colonne1, alias_colonne2, ...)]

AS subquery

WITH CHECK OPTION [CONSTRAINT constraint] ;

- la modification d'une table de base affecte la vue
- le corps d'une vue ne peut contenir de clause ORDER BY ou FOR UPDATE
- on ne peut effectuer des insertions, des mises à jours et des suppressions dans une vue contenant une jointure, des opérateurs ensemblistes, des fonctions de groupe, les clauses GROUP BY, CONNECT BY ou START WITH et l'opérateur DISTINCT.

Exemple : create view ActiveStock(CodeStock,Quantite_Stock)
as select code_stock,stock_quant from stock;;

SE : II. Le Schéma externe (les vues)

• I.3. Le Schéma externe (les vues)

• Les vues personnelles

- Une vue n'induit aucun stockage puis qu'elle n'existe pas physiquement, et permet d'obtenir une représentation différente des tables sur les quelles elle est basée.

Opérations autorisées

- Vue avec jointure : Delete : NON Update : NON Insert : NON;
- Vue avec GB ou Distinct : Delete : NON Update : NON Insert : NON
- Vue avec référence à RowNum : Delete : NON Update : NON Insert : NON;
- Vue avec une colonne issue d'une expression : Delete (sur une autre colonne) : OUI Update (sur une autre colonne) : OUI Insert : OUI
- Vue avec au moins une colonne NOT NULL absente : Delete : OUI Update : OUI Insert : NON

Les règles définissant les vues modifiables sont très strictes.

1. La vue doit être basée sur une seule table.
2. Toute colonne non référencée dans la vue doit pouvoir être mise à NULL ou disposer d'une valeur par défaut.
3. On ne peut pas mettre-à-jour un attribut qui résulte d'un calcul ou d'une opération.

CHAPITRE N° 2

LE LANGAGE PL/SQL

PL : INTRODUCTION

- Le langage PL/SQL est une extension procédurale du langage SQL.
- Contrairement au langage SQL, qui est non procédural (il est un langage ensembliste), le PL/SQL est un langage procédural qui s'appuie sur toutes les structures de programmations traditionnelles (variables, itérations, tests, séquences).
- Le PL/SQL s'intègre dans les outils SQL*FORMS, SQL*PLUS, PRO*C; il intègre des ordres SQL de gestion de la base de données :
 - SELECT
 - INSERT, UPDATE, DELETE
 - COMMIT, ROLLBACK, SAVEPOINT
 - TO_CHAR, TO_DATE, UPPER, ...

PL : I. Structure d'un bloc PL /SQL

DECLARE Déclaration de variables, constantes, exceptions, curseurs

BEGIN [nom_du_bloc]
Instructions SQL et PL/SQL

EXCEPTION
Traitement des exceptions
(gestion des erreurs)

END [nom_du_bloc] ;

- les sections DECLARE et EXCEPTION **sont facultatives**.
- chaque instruction se termine par un « ; »
- Les commentaires : sur une ligne « -- » ou « /* sur plusieurs lignes */ »

PL : II. Types de données et variables

- II.1. Les différentes types de données

TYPE	VALEURS
BINARY-INTEGER	entiers allant de -2^{31} à 2^{31})
POSITIVE / NATURAL	entiers positifs allant jusqu'à $2^{31} - 1$
NUMBER	Numérique (entre -2^{418} à 2^{418})
INTEGER	Entier stocké en binaire (entre -2^{126} à 2^{126})
CHAR (n)	Chaîne fixe de 1 à 32767 caractères (différent pour une colonne de table)
VARCHAR2 (n)	Chaîne variable (1 à 32767 caractères)
LONG	idem VARCHAR2 (maximum 2 gigaoctets)
DATE	Date (ex. 01/01/1996 ou 01-01-1996 ou 01-JAN-96 ...)
RAW	Permet de stocker des types de données binaire relativement faibles(≤ 32767 octets) idem VARCHAR2. Les données RAW ne subissent jamais de conversion de caractères lors de leur transfert entre le programme et la base de données.

PL : II. Types de données et variables

- II.1. Les différentes types de données

LONG RAW	Idem LONG mais avec du binaire
CLOB	Grand objet caractère. Objets de type long stockés en binaire (maximum 4 giga octets) Déclare une variable gérant un pointeur sur un grand bloc de caractères, mono-octet et de longueur fixe, stocké en base de données.
BLOB	Grand objet binaire. Objets de type long (maximum 4 giga octets) Déclare une variable gérant un pointeur sur un grand objet binaire stocké dans la base de données (Son ou image).
NCLOB	Support en langage nationale (NLS) des grands objets caractères. Déclare une variable gérant un pointeur sur un grand bloc de caractères utilisant un jeu de caractères mono-octets, multi-octets de longueur fixe ou encore multi-octets de longueur variable et stocké en base de données.
ROWID	Composé de 6 octets binaires permettre d'identifier une ligne par son adresse physique dans la base de données.
UROWID	Le U de UROWID signifie Universel, une variable de ce type peut contenir n'importe quel type de ROWID de n'importe quel type de table.
BOOLEAN	Bouléen, prend les valeurs TRUE, FALSE, NULL

PL : II. Types de données et variables

- **II.2. Les variables et les constantes**

- La déclaration d'une variable ou d'une constante se fait dans la partie DECLARE d'un programme PL/SQL.
- On peut déclarer le type d'une variable d'une façon implicite ou explicite.
- **Les différents types de variables locales**
 - Variables de types ORACLE
 - Variables de type BOOLEAN
 - Variables faisant référence au dictionnaire de données

PL : II. Types de données et variables

- **II.2. Les variables et les constantes**

- **Variables de type ORACLE**

Syntaxe :

```
nom_var TYPE_ORACLE; /*Explicite*/  
nom_var BOOLEAN; /*Implicite */
```

Exemple :

```
DECLARE  
    nom      CHAR(20);  
    prenom   CHAR(15);  
    retour   BOOLEAN;  
  
BEGIN  
    ...  
END;
```

PL : II. Types de données et variables

- **II.2. Les variables et les constantes**
- **Variables faisant référence au dictionnaire de données (implicite)**

Syntaxe :

nom_var **table . Colonne % TYPE** ; /*Meme type qu'un attribut de table*/

nom_var **table % ROWTYPE** ; /*Meme structure qu'une ligne de table*/

nom_var2 **nom_var1 % TYPE**; /*Meme type qu'une autre variable*/

Exemple :

```
DECLARE
```

```
    nom      Clients.nomcli%TYPE;
```

```
    ligne    Clients%ROWTYPE;
```

```
    ancien_stock NUMBER(5);
```

```
    nouveau_stock ancien_stock%TYPE;  --NUMBER(5)
```

```
BEGIN
```

```
...
```

```
END;
```

La structure ligne contient autant de variables que de colonnes de la table. Ces variables portent le même nom et sont de même type que les colonnes de la table.

PL : II. Types de données et variables

- **II.2. Les variables et les constantes**
- Initialisation de variables

Syntaxe :

```
opérateur :=  
SELECT ... INTO ...  
DEFAULT X
```

Exemple :

```
DECLARE  
    var1 CHAR(10) := 'TRAORE';  
    var2 NUMBER(5,2) := 100;  
    var3 CHAR(10);  
    var4 DATE;  
    var5 DEFAULT 0;  
  
BEGIN  
    SELECT col1, col2  
    INTO var3, var4  
    FROM ... ;  
  
    ...  
END;
```

PL : III. Les structures

- **III.1. Structure conditionnelle (IF)**

Syntaxes :

```
IF <Condition>  
  THEN <Instructions>;  
    ELSEIF <Condition>  
      THEN <Instructions>;  
    ELSE <Instructions>;  
END IF;
```

PL : III. Les structures

- **III.1. Structure conditionnelle (CASE)**

Syntaxe 1 :

```
CASE <Expression>  
    WHEN <expression_comparaison> THEN <expression_résultat>  
    [ ... ]  
    [ ELSE <expression_résultat_par_défaut> ]  
END
```

Syntaxe 2 :

```
CASE  
    WHEN <condition> THEN <expression_résultat>  
    [ ... ]  
    [ ELSE <expression_résultat_par_défaut> ]  
END CASE
```

l'instruction CASE génère une exception si aucune condition n'est vérifiée et qu'il n'y a pas de ELSE. S'il est normal qu'aucune condition ne soit vérifiée, et qu'il n'y a rien à faire dans ce cas là, il est possible de mettre une clause ELSE NULL; (l'instruction NULL ne faisant justement rien).

PL : III. Les structures

- **III.2. Structure Répétitive (LOOP)**

Syntaxe :

BEGIN

LOOP [label]
 <instructions>;
EXIT WHEN.....

END LOOP [label];

END;

Exécution d'un traitement plusieurs fois, le nombre n'étant pas connu mais dépendant d'une condition.

Pour sortir de la boucle, utiliser la clause :

EXIT [label] WHEN condition

l'instruction LOOP peut cohabiter avec les traditionnelles WHILE et FOR.

PL : III. Les structures

- **III.2. Structure Répétitive (FOR)**

Syntaxe :

```
BEGIN
    FOR <indice> IN [REVERSE] exp1 ... exp2
    LOOP
        <instructions>;
    END LOOP;
END;
```

Exécution d'un traitement un certain nombre de fois. Le nombre étant connu.

- inutile de déclarer indice
- indice varie de exp1 à exp2 de 1 en 1

PL : III. Les structures

- **III.2. Structure Répétitive (WHILE)**

Syntaxe :

```
BEGIN
    WHILE <condition>
    LOOP
        <instructions>;
    END LOOP;
END;
```

Exécution d'un traitement tant qu'une condition reste vraie.

PL : III. Les structures

- **III.3. Les instructions GOTO, NULL**

Syntaxes :

BEGIN

GOTO <Nom_Etiquette> ;

<<Nom_Etiquette>>

<instructions>

END;

BEGIN

NULL;

END;

L'instruction GOTO effectue un branchement sans condition sur une autre commande de la même section d'exécution d'un bloc PL/SQL ; Null permet de dire au système de ne "rien faire".

Exemple:

set serveroutput on

BEGIN

GOTO deuxieme_affichage;

DBMS_OUTPUT.PUT_LINE ('Cette ligne ne sera jamais affichée') ;

<<deuxieme_affichage>>

DBMS_OUTPUT.PUT_LINE ('Cette ligne sera toujours affichée') ;

END ; /

PL : III. Les structures

• III.4. Gestion de l’Affichage

- Activation du package **DBMS_OUTPUT** ;;Commande : ***set serveroutput on.***
- Utilisation de la fonction ***put_line*** du package.

Syntaxes :

```
set serveroutput on
```

```
...
```

```
DECLARE
```

```
...
```

```
BEGIN
```

```
    DBMS_OUTPUT . put_line(' ');
```

```
END;
```

Exemple:

```
set serveroutput on
```

```
DECLARE
```

```
message varchar2(100);
```

```
BEGIN
```

```
message := 'Essai d"affichage';
```

```
DBMS_OUTPUT.put_line ('Test : ' || message);
```

```
END;
```

```
/
```

TP. Les structures

- **Structure Répétitive LOOP (loop, while-loop, for-loop)**

```
BEGIN
    LOOP [label]
        <instructions>;
    EXIT WHEN.....
    END LOOP [label];
END;
```

```
BEGIN
    WHILE <condition>
    LOOP
        <instructions>;
    END LOOP;
END;
```

```
BEGIN
    FOR <indice> IN [REVERSE] exp1 ... exp2
    LOOP
        <instructions>;
    END LOOP;
END;
```

TP : à l'aide du sql, créer une table "result" et insérez y les 10 premiers chiffres à l'aide du PLSQL

Programme pour afficher le type du produit

- afficher 'ceci est un' (ce que l'on cherche réellement) en fonction du type de produit
- Insérée dans la table type produits, des types 1, téléphones 2 – téléviseurs 3 ordinateur

Démarche

- Début
- POUR i Dans (select....)
- Répéter
- Cas i.(ceque l'on cherche)
- Quand 1 afficher 'ceci est un' (ceque l'on cherche réellement)
- Quand 2...
-
- Fin

Syntaxe 1 :

```
CASE <Expression>  
    WHEN <expression_comparaison> THEN <expression_résultat>  
    [ ... ]  
    [ ELSE <expression_résultat_par_défaut> ]  
END
```

PL : IV. Les Curseurs

• IV.1. Définitions

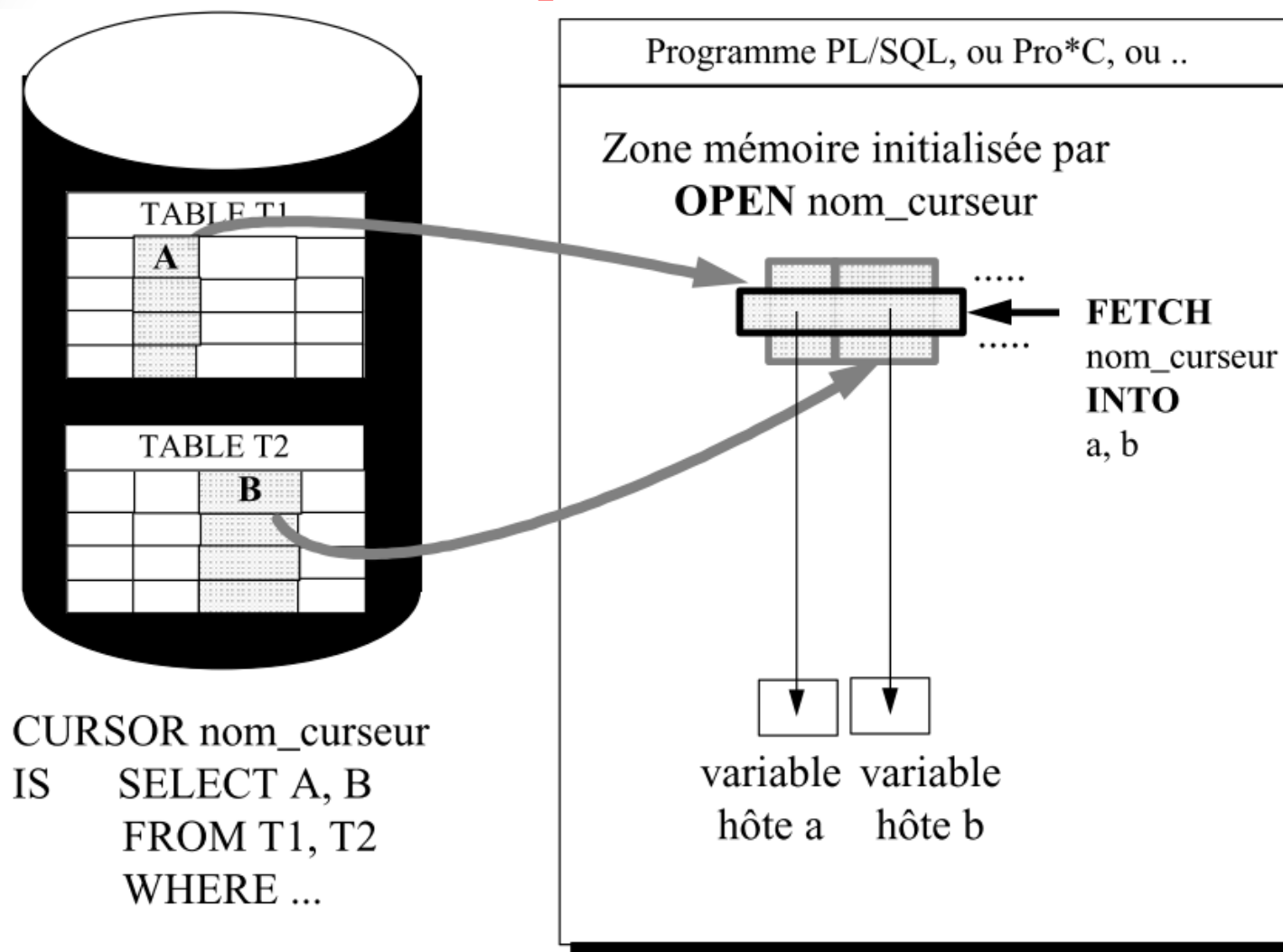
- **Un curseur est une variable qui pointe vers le résultat d'une requête SQL.** La déclaration du curseur est liée au texte de la requête.
- **Un curseur permet de lire séquentiellement le résultat d'une requête.** On peut ainsi traiter les lignes résultantes une par une en déchargeant les données lues dans des variables hôtes.

Il existe 2 types de curseurs :

- **CURSEUR EXPLICITE :**
Curseur SQL généré et géré par l'utilisateur pour traiter un ordre SELECT qui ramène plus d'une ligne.
- **CURSEUR IMPLICITE :**
Curseur SQL généré et géré par le noyau pour chaque ordre SQL d'un bloc.

PL : IV. Les Curseurs

- IV.1. Les Curseurs explicites



PL : IV. Les Curseurs

• IV.1. Les Curseurs explicites

▪ Les Quatre étapes de définition d'un curseur

- déclaration du curseur <DECLARE> : Indication du nom du curseur et l'ordre SQL associé, Syntaxe : **DECLARE**

CURSOR <nom_curseur> **IS** <ordre_select> ;

- ouverture du curseur <OPEN> : Exécution de la requête, allocation mémoire pour y stocker le résultat, positionnement du curseur sur le premier enregistrement.

Syntaxe : **BEGIN**

OPEN <nom_curseur> ;

- traitement des lignes <FETCH> : Lecture de la zone pointée par le curseur, affectation des variables hôtes, passage à l'enregistrement suivant. Après l'exécution du SELECT les lignes ramenées sont traitées une par une, la valeur de chaque colonne du SELECT doit être stockée dans une variable réceptrice.

Syntaxe : **FETCH** <nom_curseur> **INTO** liste_variables ;

- fermeture du curseur <CLOSE> : Libération de la zone mémoire allouée pour le résultat de la requête.

Syntaxe : **CLOSE** <nom_cursseur>;

PL : IV. Les Curseurs

- **IV.1. Les Curseurs explicites**

- Syntaxe globale

DECLARE

```
CURSOR <nom_cursseur> IS SELECT <nom_attribn>, <nom_attribn>  
FROM <Nom_tablen> WHERE <Condition>;
```

BEGIN

```
OPEN <nom_cursseur>;
```

LOOP

```
    FETCH<nom_cursseur> INTO <nom_attrib1>, <Nom_attrib2>;  
    EXIT WHEN <Conditionn>;
```

```
END LOOP;
```

```
    CLOSE<nom_cursseur>;
```

END;

PL : IV. Les Curseurs

- **IV.1. Les Curseurs explicites**

- Attribut sur les curseurs

Chaque curseur possède quatre attributs :

- **%FOUND** Génère un booléen VRAI lorsque le FETCH réussi (données lues)
- **%NOTFOUND** Inverse de %FOUND (généralement plus utilisé que %FOUND)
- **%ISOPEN** Génère un booléen VRAI lorsque le curseur spécifié en argument est ouvert.
- **%ROWCOUNT** Renvoie le nombre de lignes contenues.

Chaque attribut s'utilise en étant préfixé par le nom du curseur :

`<nom_curseur>%ATTRIBUT`

PL : IV. Les Curseurs

- **IV.1. Les Curseurs explicites**

Les Quatre étapes de définition d'un curseur

EMPLOYE

```
Emp_id integer not null ,  
Emp_nom varchar2(30) not null,  
Emp_salaire number(8) not null,  
Emp_emploi varchar2(20) null,  
Emp_Emp_id integer null
```

- Crée la table cadres

```
SQL> desc cadres
```

Nom	NULL ?	Type
EMP_NOM		VARCHAR2(30)
EMP_SALAIRE		NUMBER

- Faites un programme à l'aide d'un curseur parcourant la table employé, copie les employés dont le salaire est supérieur à 100000, et les insert dans la table cadre que vous avez créé (NB : la table cadres à une clef numérique automatique).

PL : IV. Les Curseurs

• IV.1. Les Exceptions

Il existe deux types d'exception :

interne,
externe.

- **Les exceptions internes** sont générées par le moteur du système (division par zéro, connexion non établie, table inexistante, privilèges insuffisants, mémoire saturée, espace disque insuffisant, ...).
- **Les exceptions externes** sont générées par l'utilisateur (stock à zéro, ...).
- Le gestionnaire des exceptions du PL/SQL ne sait gérer que des erreurs nommées (noms d'exceptions). Par conséquent, toutes les exceptions doivent être nommées et manipulées par leur nom.
- **Les erreurs ORACLE** générées par le noyau sont numérotées (ORA-xxxxx). Il a donc fallu établir une table de correspondance entre les erreurs ORACLE et des noms d'exceptions.
- **A chaque erreur ORACLE correspond un code SQL (SQLCODE)** que l'on peut tester dans le langage hôte (PL/SQL, Pro*C, etc. ...).

PL : IV. Les Curseurs

- **IV.1. Les Exceptions**

exemples d'exceptions prédéfinis et des codes correspondants

Nom d'exception	Erreur ORACLE	SQLCODE
CURSOR_ALREADY_OPEN	ORA-06511	-6511
DUP_VAL_ON_INDEX	ORA-00001	-1
INVALID_CURSOR	ORA-01001	-1001
INVALID_NUMBER	ORA-01722	-1722
LOGIN_DENIED	ORA-01017	-1017
NO_DATA_FOUND	ORA-01403	+100
PROGRAM_ERROR	ORA-06501	-6501
ROWTYPE_MISMATCH	ORA-06504	-6504
TIMEOUT_ON_RESOURCE	ORA-00051	-51
TOO_MANY_ROWS	ORA-01422	-1422
ZERO_DIVIDE	ORA-01476	-1476

PL : IV. Les Curseurs

- **IV.1. Les Exceptions**

Exception ORACLE dont le nom est prédéfini

Il suffit de préciser le traitement dans le module EXCEPTION. Aucune déclaration n'est nécessaire.

Exemple de gestion de l'exception prédéfinie NO_DATA_FOUND ,
DUP_VAL_ON_INDEX, NO_DATA_FOUND

- ...
- OTHERS :

DECLARE

...

BEGIN

EXCEPTION

WHEN NO_DATA_FOUND THEN

<traitement>;

END;

PL : IV. Les Curseurs

• IV.1. Les Exceptions

Exception ORACLE dont le nom n'est pas prédéfini

Il va falloir créer une correspondance entre le code erreur ORACLE et le nom de l'exception. Ce nom est choisi librement par le programmeur.

Un nommage évocateur est largement préconisé et il faut éviter les noms comme erreur 1023, ...;

DECLARE

`<nom_erreur> EXCEPTION; /*on donne un nom à l'erreur*/`

`PRAGMA EXCEPTION_INIT (<nom_erreur>, <code_erreur>) /*on associe le
nom de l'erreur à un code erreur*/`

...

BEGIN

`... /*l'erreur Oracle est détectée par le système*/`

EXCEPTION

`WHEN nom_erreur THEN ... <traitement de l'erreur> ;`

END;

NB:

- **SQLCODE** renvoie le code de l'erreur courante (numérique)
- **SQLERRM[(code_erreur)]** renvoie le libellé de l'erreur courante ou le libellé de l'erreur dont le numéro est passé en paramètre.

PL : V. Triggers

Un trigger permet de spécifier les réactions du système d'information lorsque l'on « touche » à ses données. Concrètement il s'agit de définir un traitement (un bloc PL/SQL) à réaliser lorsqu'un événement survient. Les événements sont de six types (dont trois de base) et ils peuvent porter sur des tables ou des colonnes :

BEFORE INSERT

AFTER INSERT

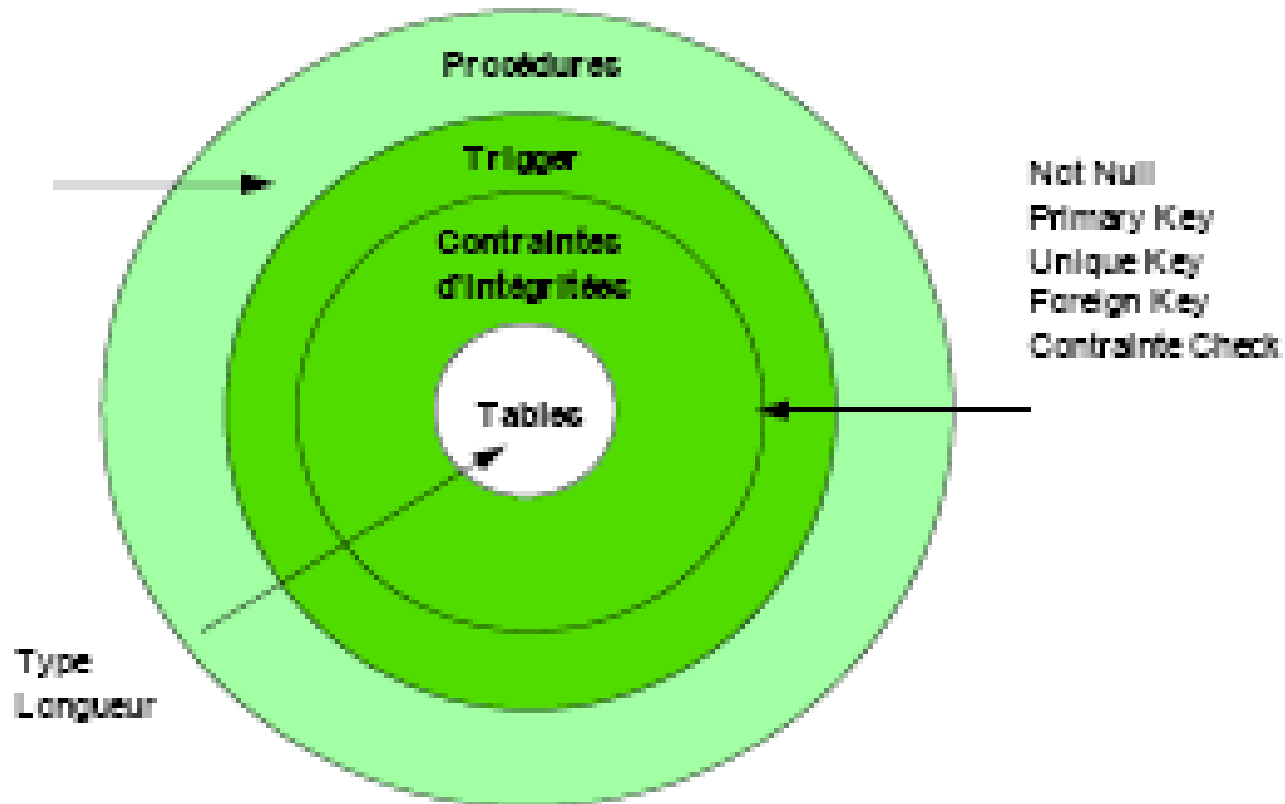
BEFORE UPDATE

AFTER UPDATE

BEFORE DELETE

AFTER DELETE

Select
Insert
Update
Delete
Etc...



PL : V. Triggers

Activer un trigger

Un trigger peut être activé ou désactivé respectivement par les clauses ENABLE et DISABLE.

```
SQL> alter trigger modif_voyage disable;  
Trigger altered.
```

Supprimer un Trigger

Comme pour tout objet ORACLE :

```
SQL> drop trigger modif_voyage;  
Trigger dropped.
```

il est possible de créer des triggers sur les vues en utilisant le mot clé : INSTEAD.

```
create or replace trigger tr_voyage  
instead of delete on v_voyage  
for each row  
begin  
delete from voyage  
where id_voyage = :old.id_voyage;  
dbms_output.put_line('le trigger fonctionne !!!');  
end;  
/
```

PL : V. Triggers

Ils se déclenchent sur un événement tels que :

After startup

Before shutdown

After servererror

After logon

Before logoff

{before|after} alter

{before|after} analyse

{before|after} {audit|noaudit}

{before|after} {comment|DDL|drop|rename|truncate}

{before|after} create

{before|after} grant

{before|after} revoke

PL : VI. Procédures, Fonctions et Packages

- **Définitions**

- Une **procédure** est une unité de traitement qui contient des commandes SQL relatives au langage de manipulation des données, des instructions PL/SQL, des variables, des constantes, et un gestionnaire d'erreurs.
- Une **fonction** est une procédure qui retourne une valeur.
- Un **package** est un agrégat de procédures et de fonctions.
Les packages, procédures, ou fonctions peuvent être appelés depuis toutes les applications qui possèdent une interface avec ORACLE (SQL*PLUS, Pro*C, SQL*Forms, ou un outil client particulier comme NSDK par exemple).

Les packages permettent de regrouper des procédures ou des fonctions (ou les deux). On évite ainsi d'avoir autant de sources que de procédures.

PL : VI. Procédures, Fonctions et Packages

• VI.1. Procédures

```
CREATE OR REPLACE PROCEDURE <nom_procedure>  
(liste d'arguments en INPUT ou OUTPUT) IS  
/*déclaration de variables, de constantes, ou de curseurs*/  
BEGIN  
...  
EXCEPTION  
...  
END;  
/
```

Au niveau de la liste d'arguments :

- les arguments sont précédés des mots réservés IN, OUT, ou IN OUT,
- ils sont séparés par des virgules,
- le mot clé IN indique que le paramètre est passé en entrée,
- le mot clé OUT indique que le paramètre est modifié par la procédure,
- on peut cumuler les modes IN et OUT (cas du nombre de lignes demandées à une procédure d'extraction).

PL : VI. Procédures, Fonctions et Packages

• VI.1. Procédures

Exemple de procédure qui modifie le salaire d'un employé.

Arguments : Identifiant de l'employée, Taux

```
CREATE PROCEDURE modifie_salaire (id in number, taux in number) IS  
begin  
    update employe set salaire=salaire*(1+taux)  
    where Id_emp= id;  
EXCEPTION  
    when no_data_found then  
        raise_application_error (-20010,'Employé inconnu :'|to_char(id));  
end;/
```

Compilation de la procédure modifie salaire

```
SQL> start modifie_salaire      modifie_salaire (15,-0.5);  
Procedure created.            end;
```

Appel de la procédure modifie salaire

```
SQL> begin  
modifie_salaire (15,-0.5);  
end;
```

Lancement du script démarre.sql :

```
SQL> start démarre
```


PL : VI. Procédures, Fonctions et Packages

- **VI.2. Fonctions**

Exemple de fonction qui modifie le salaire d'un employé.

Arguments : Identifiant de l'employée, Taux

create or replace function nom_fonction

(liste d'arguments en INPUT ou OUTPUT)

return type_valeur_retour

is

/*Déclaration

déclaration de variables, de constantes, ou de curseurs

begin

...

return (valeur);

exception

...

end;

/

PL : VI. Procédures, Fonctions et Packages

• VI.2. Fonctions

Exemple de fonction qui retourne la moyenne des salaires des employés par bureau (regroupé par responsable du bureau) .

- Argument : Identifiant de l'employé
- Retour : moyenne du bureau

```
create or replace function moyenne_salaire (v_id_employe IN NUMBER)
return NUMBER
is
valeur NUMBER;
begin
select avg(salaire)
into valeur
from employe
groupe by emp_id_emp
having emp_id_emp=v_id_emp;
return (valeur);
end;
/
```

PL : VI. Procédures, Fonctions et Packages

• VI.2. Fonctions

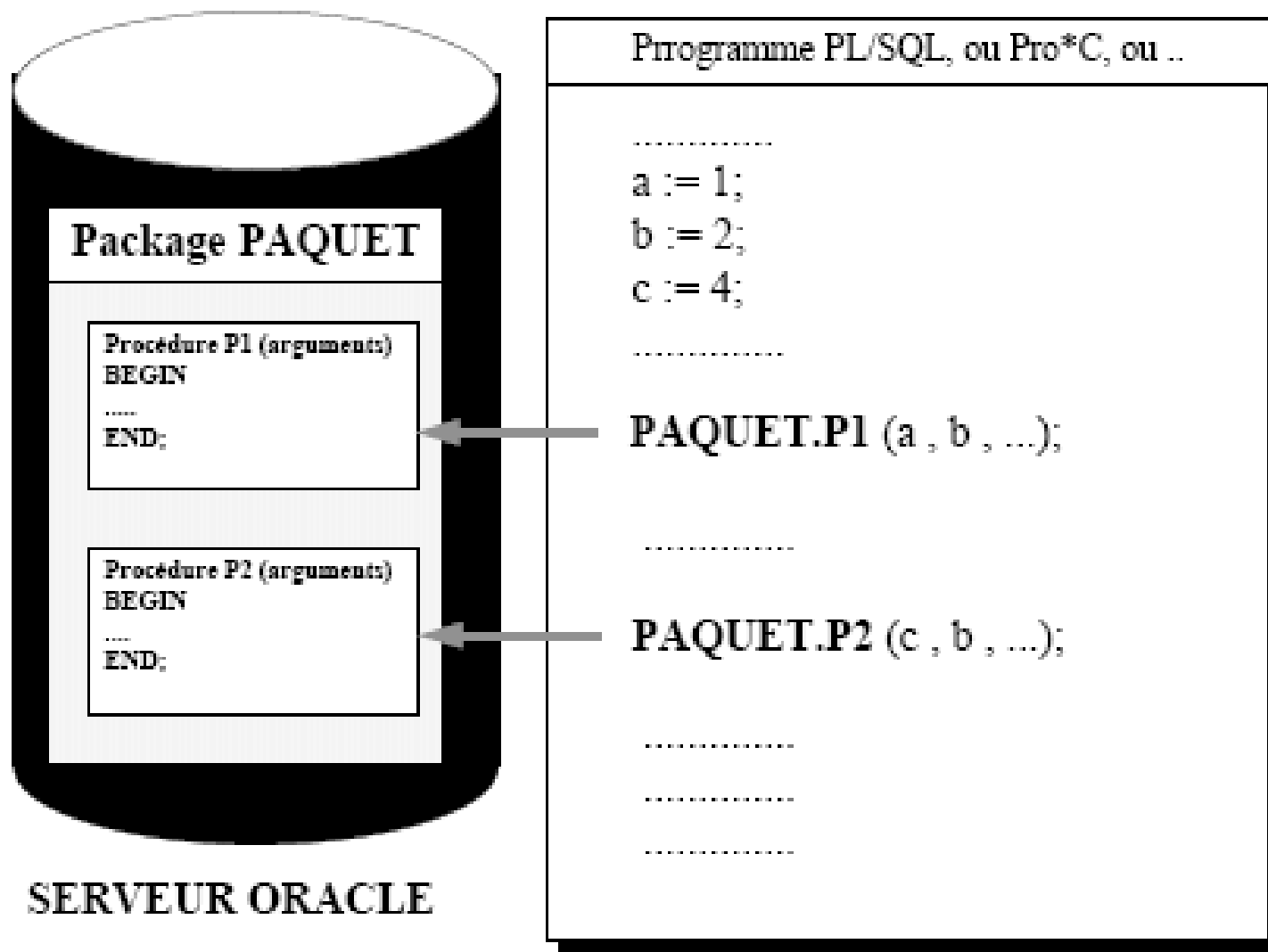
Exemple de fonction qui retourne la moyenne des salaires des employés par bureau (regroupé par responsable du bureau) .

- Argument : Identifiant de l'employé
- Retour : moyenne du bureau

```
create or replace function moy_sal(v_id_employe IN NUMBER)
return NUMBER
IS valeur NUMBER;
begin
    select avg(emp_salaire) into valeur from employe
    group by emp_id
    having emp_id = v_id_employe;
return (valeur);
end;
/
select moy_sal(1) from dual;
```

PL : VI. Procédures, Fonctions et Packages

- **VI.3. Packages**



Principe général d'utilisation d'un package

FIN

EXOS

- Créez un trigger qui supprime la commande lorsque le client est supprimé
- créez une fonction qui calcul la moyenne des salaires

- EXEC SQL UPDATE employees SET salary = 1.10 * salary
- WHERE department_id = :department_id;

Example 13-1 Changing PLSQL_OPTIMIZE_LEVEL

Create two procedures:

```
CREATE OR REPLACE PROCEDURE p1 AUTHID DEFINER AS
BEGIN
NULL;
END;/
```

```
CREATE OR REPLACE PROCEDURE p2 AUTHID DEFINER AS
BEGIN
NULL;
END;/
```

Display the optimization levels of the two procedures:

```
SELECT NAME, PLSQL_OPTIMIZE_LEVEL
FROM USER_PLSQL_OBJECT_SETTINGS
WHERE NAME LIKE 'P%' AND TYPE='PROCEDURE'
ORDER BY NAME;
```


PL : V. Triggers

- Exemple: Suppression d'un client
→ On supprime toutes ses commandes

```
CREATE TRIGGER suppclient
BEFORE DELETE ON CLIENT
FOR EACH ROW
BEGIN
    DELETE FROM COMMANDE WHERE Codcli= :OLD.Codcli
END;
/
```